

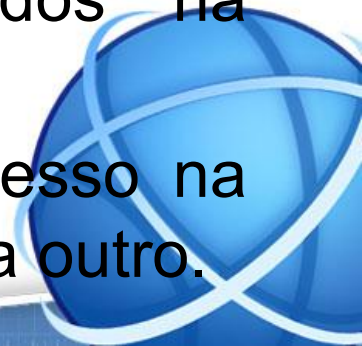
Software Básico



**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
NORTE DE MINAS GERAIS**

Tradução e execução de programas

- Quando um programa é colocado em execução, o sistema operacional é ativado para encontrar um espaço de memória onde este programa irá residir durante a execução.
 - Processo é criado;
 - Os endereços interno do programa são virtuais;
 - Quando o sistema de computação usa memória virtual;
 - Páginas do programa são carregados na memória;
- A organização das informações de um processo na memória varia de um sistema operacional para outro.

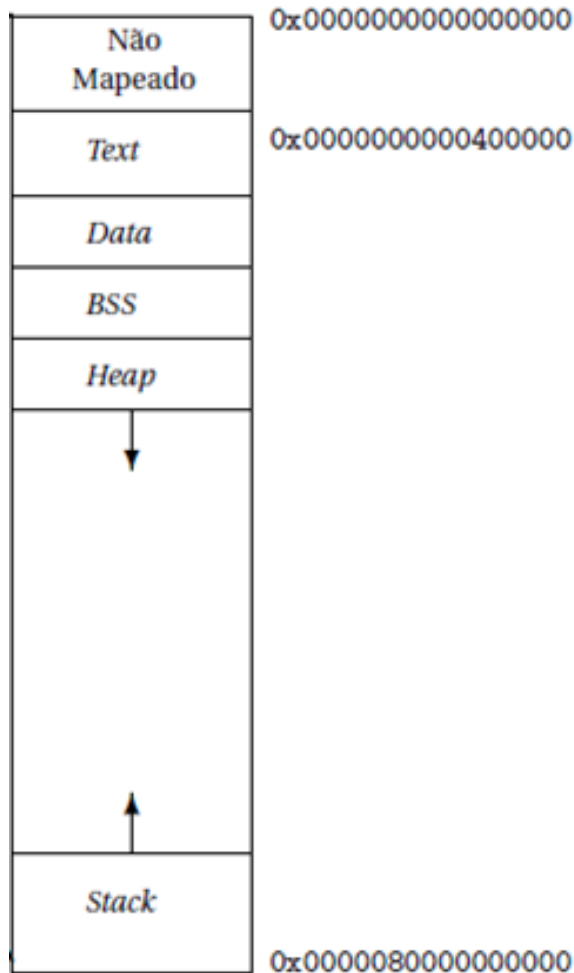


Tradução e execução de programas

- Um **programa** em execução recebe uma grande quantidade de **memória virtual**.
 - Utiliza-se um mecanismo para mapear os endereços da memória virtual para a memória física, essa técnica é chamado **paginação**;
- **Espaço de endereçamento virtual**:
 - Mesmo podendo endereçar até **64 bits** (um total de 2^{64} endereços), somente 2^{48} endereços são utilizados:
 - (0x00000000000000000000 até 0x00007fffffffffff).



Tradução e execução de programas

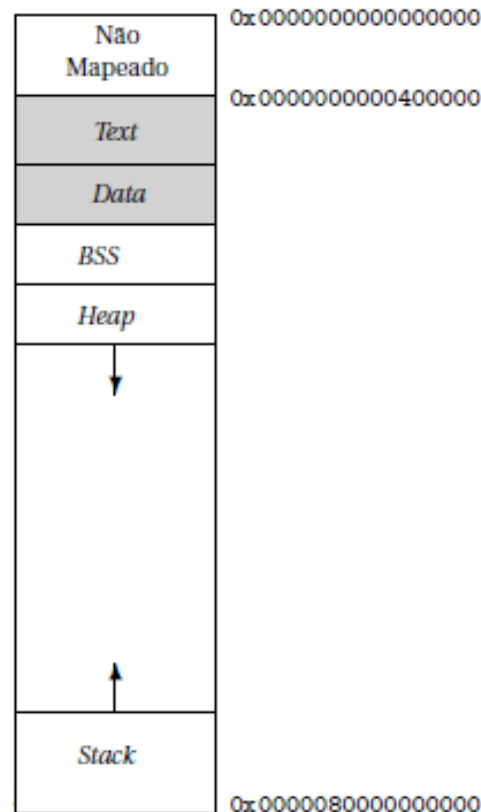


- O **espaço de endereçamento** é muito maior do que a quantidade de **memória física**.
- O **espaço virtual** de **endereçamento** de um programa em execução no modelo ELF (*Executable and Linkable Format*) do Linux;



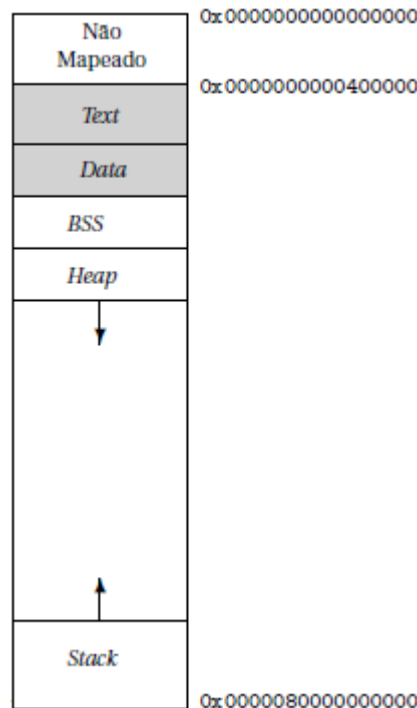
Tradução e execução de programas

- Segmento de processo é dividido em partes, ou seções, chamadas seção **text**, **data**, **bss**, **heap** e **stack**.



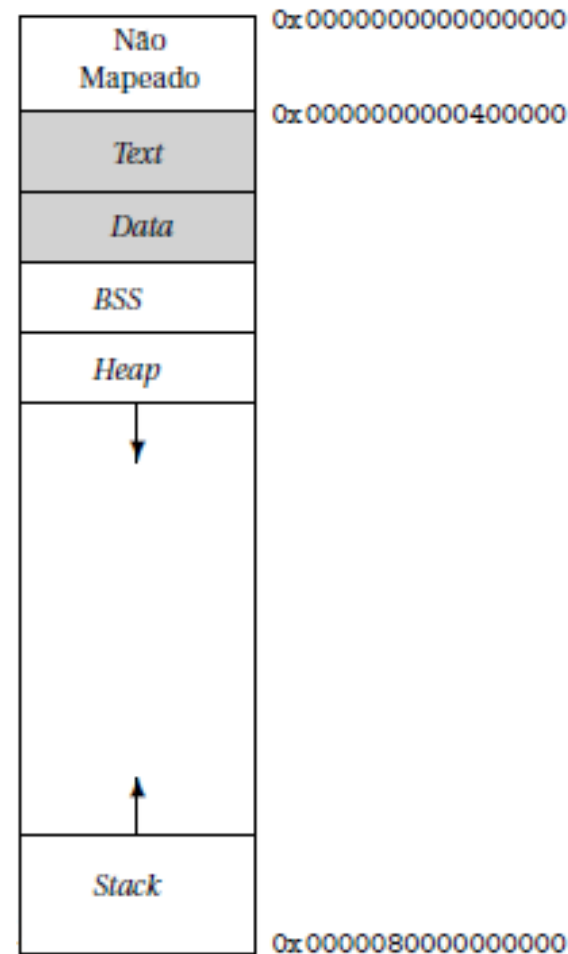
Esqueleto de programa em Assembly

- **Seção de Código e de Dados**
 - A **seção *text*** contém as **instruções** do programa, enquanto que a **seção *data*** contém as variáveis globais.



Esqueleto de programa em Assembly

- **Seção de Código e de Dados**
 - A **seção *text*** inicia no endereço virtual 0x0000000000400000, mas não tem um **endereço de término** indicado;
 - A **seção *data*** não tem um endereço de início e de fim especificados;



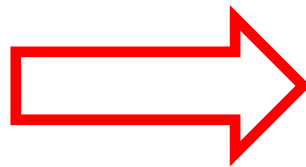
Esqueleto de programa em Assembly

- Programas em **assembly** são **divididos em seções**, e o programa apresentado contém **duas seções**.
 - A **.section .data** contém as variáveis globais do programa (como este programa não usa variáveis globais, esta seção está vazia)
 - A **.section .text** contém os comandos a serem executados quando o programa for colocado em execução.



Esqueleto de programa em Assembly

```
int main()  
{  
    return 0;  
}
```



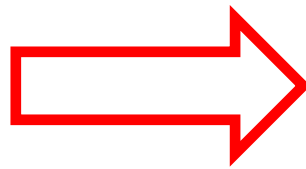
```
1 .section .data  
2 .section .text  
3 .globl _start  
4 _start:  
5 movq $60, %rax  
6 movq $13, %rdi  
7 syscall
```

- O rótulo **_start** é especial e deve sempre estar presente em um programa **assembly**.
- Ele corresponde ao **endereço da primeira instrução** do programa que será executada, e deve ser declarada como **globl** (linha 3).



Esqueleto de programa em Assembly

```
int main()  
{  
    return 0;  
}
```



```
1 .section .data  
2 .section .text  
3 .globl _start  
4 _start:  
5 movq $60, %rax  
6 movq $13, %rdi  
7 syscall
```

- O rótulo **_start** indica o local onde a execução do programa deve iniciar.



Execução do programa

- **as** exemplo1.s -o exemplo1.o
 - **as**: *The portable GNU assembler*
 - converte um programa escrito em *assembly* (**fonte.s**) num arquivo objeto (**fonte.o**). Um **arquivo objeto** é uma versão incompleta de um arquivo executável.
- **ld** exemplo1.o -o exe
 - **ld**: *The GNU linker*
 - **ld** combina arquivos **.objeto** e gera um arquivo executável.
- O comando **./exe** invoca o sistema operacional (**loader**) que o coloca em execução.



GNU Binutils

- The GNU ***Binutils*** are a collection of binary tools. The main ones are:
 - **ld** - the GNU linker.
 - **as** - the GNU assembler.
- **Instalação:**
 - `sudo apt-get update -y`
 - `sudo apt-get install -y binutils-common`



Esqueleto de programa em Assembly

```
1 long int a, b;  
2 int main ( );  
3 {  
4     a=7;  
5     b=7;  
6     a = a+b;  
7     return a;  
8 }
```

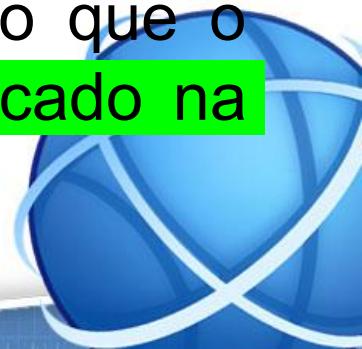
```
.section .data  
a: quad 0  
a: quad 0  
.section .text  
.globl _start  
_start:  
    movq $7, a  
    movq $7, b  
    movq a, %rax  
    movq b, %rbx  
    addq %rax, %rbx  
    movq %rbx, a  
    movq a, %rdi  
    movq $60, %rax  
    syscall
```



Chamadas ao sistema Linux

```
...  
12 movq $60, %rax  
13 movq %rbx, %rdi  
14 syscall
```

- A chamada ao sistema é executada pelo comando ***syscall***. Os dois comandos anteriores são os parâmetros desta chamada.
- O valor em %rax indica qual serviço é pedido (terminar a execução do programa) enquanto que o valor em %rdi indica qual o valor a ser colocado na variável de ambiente \$?.



Linguagem Assembly

- A linguagem de montagem (*Assembly*) x86-64 possui dois padrões de programação:
 - **Sintaxe Intel**
 - Dominante no sistema operacionais *Microsoft Windows*;
 - **Sintaxe AT&T**
 - Dominante nos sistemas operacionais *Unix* e *Linux*;



Linguagem Assembly

- # para comentário
- **Valor imediato:**
 - *\$valor*
 - Valor decimal: \$15
 - Valor hexadecimal: \$0x0A
- **Constante:**
 - decimal 15
 - hexadecimal 0x0A



Linguagem Assembly

- **Declaração de Variáveis globais**
 - as variáveis não estão associados a nenhum **tipo**.
 - **Sintaxe:**

<Identificador> : .<quantidade de bytes> <valor>

.section .data

a: .byte #(8 bits)
b: .word #(16 bits)
c: .long #(32 bits)
d: .quad #(64 bits)

Observação: <valor> é opcional



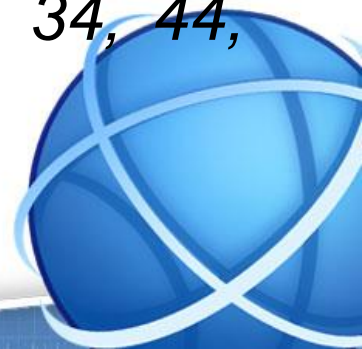
Linguagem Assembly

- **Declaração de vetor global**

- A forma de criar um vetor com valores fixos. Basta listá-los lado a lado na seção data, ao lado do rótulo associado àquele vetor
- **<Identificador> : .<quantidade de bytes>**
<valores separados por vírgulas>

.section .data

***vetorA: .quad 3, 67, 34, 222, 45, 75, 54, 34, 44,
33, 22, 66***



Acessando os dados - operandos

- A maioria das instruções tem um ou mais operandos, especificando os valores de origem para a realização de uma operação e o local de destino para armazenar o resultado.
- Os valores podem ser obtidos em uma **constantes**, **registradores** ou da **memória**. Os resultados podem ser armazenados em **registradores** ou na **memória**.



Acessando os dados - operandos

- **Modelo de endereçamento:**
 - Os operandos de uma instrução em *Assembly* podem variar de acordo com o local em que o dado se encontra. Como exemplo, observe a diferença entre as instruções
 - movq **\$0**, %rbx (***endereçamento imediato***);
 - movq %rax, %rbx (***endereçamento registrador***)
 - movq A, %rbx (***endereçamento direto***);
 - movq (%rbx) , %rax (***endereçamento indireto***)
 - movq A(,%rdi,4), %rbx. (***endereçamento indexado***)



Acessando os dados - imediatos

1. Imediato:

- A constante - imediato - em **assembly** no formato **ATT** são escritos com um \$ seguido por um inteiro, **por exemplo, \$-577 ou \$0x1F.**

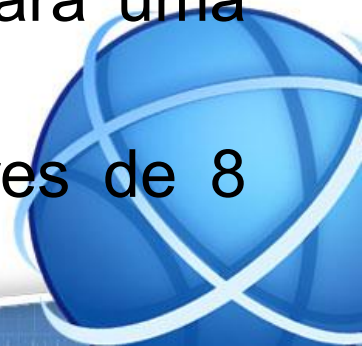
Type	Form	Operand value	Name
Immediate	$\$Imm$	Imm	Immediate
Register	E_a	$R[E_a]$	Register
Memory	Imm	$M[Imm]$	Absolute
Memory	(E_a)	$M[R[E_a]]$	Indirect
Memory	$Imm(E_b)$	$M[Imm + R[E_b]]$	Base + displacement
Memory	(E_b, E_i)	$M[R[E_b] + R[E_i]]$	Indexed
Memory	$Imm(E_b, E_i)$	$M[Imm + R[E_b] + R[E_i]]$	Indexed
Memory	$(, E_i, s)$	$M[R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(, E_i, s)$	$M[Imm + R[E_i] \cdot s]$	Scaled indexed
Memory	(E_b, E_i, s)	$M[R[E_b] + R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(E_b, E_i, s)$	$M[Imm + R[E_b] + R[E_i] \cdot s]$	Scaled indexed



Acessando os dados - registradores

2. Registrador:

- O segundo tipo de operando, **registrador**, representa o valor contido em um dos registradores:
 - de 64 *bits* para uma operação com **palavra quádrupla**,
 - de 32 *bits* para uma operação com **palavra dupla**,
 - um dos oito registradores de 16 *bits* para uma operação de **palavra**, ou
 - um dos oito elementos dos registradores de 8 *bits* para uma operação de **byte**.



Acessando os dados - registradores

E_a = indica o registrador a ;

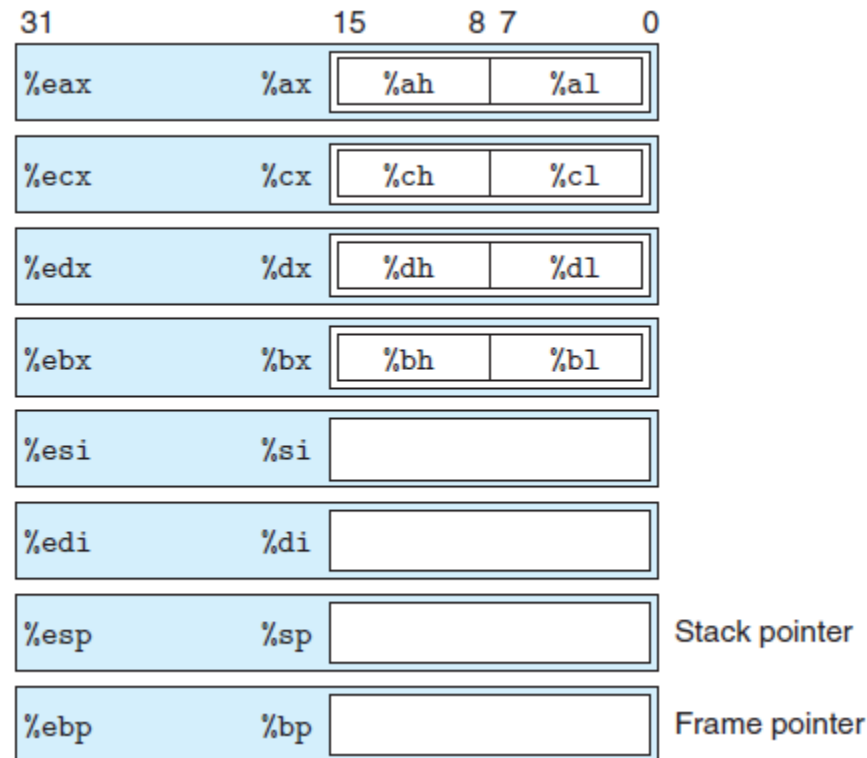
$R[E_a]$ = indica o valor contido no registrador a ;

Type	Form	Operand value	Name
Immediate	$\$Imm$	Imm	Immediate
Register	E_a	$R[E_a]$	Register
Memory	Imm	$M[Imm]$	Absolute
Memory	(E_a)	$M[R[E_a]]$	Indirect
Memory	$Imm(E_b)$	$M[Imm + R[E_b]]$	Base + displacement
Memory	(E_b, E_i)	$M[R[E_b] + R[E_i]]$	Indexed
Memory	$Imm(E_b, E_i)$	$M[Imm + R[E_b] + R[E_i]]$	Indexed
Memory	$(, E_i, s)$	$M[R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(, E_i, s)$	$M[Imm + R[E_i] \cdot s]$	Scaled indexed
Memory	(E_b, E_i, s)	$M[R[E_b] + R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(E_b, E_i, s)$	$M[Imm + R[E_b] + R[E_i] \cdot s]$	Scaled indexed



Acessando os dados - registradores

A Unidade de Processamento Central (CPU) do IA32 contém um conjunto de **oito registradores** com 32 *bits* de armazenamento.



Acessando os dados - registradores

- **Registradores *AMD64***

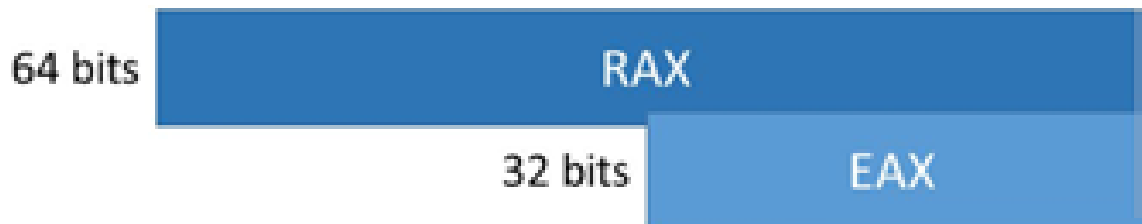
- Na arquitetura **x64** (ou também chamada de **AMD64** ou **Intel64**), foi incrementada com a expansão dos registradores já usados na arquitetura anterior x86, de 32 *bits*, os conhecidos EAX, EBX, ECX, EDC, ESI e EDI,
- Os registradores do **AMD64** são todos de 64 *bits*;
- Compatibilidade com sistema 32 *bits* (x86);



Acessando os dados - registradores

- Registradores *AMD64*

- Há uma diferença no nome deles: os de **32 bits** são precedidos pela letra **E** enquanto que os de **64 bits** são precedidos pela letra **R**.
- Assim, **%eax** é um registrador de **32 bits** enquanto que **%rax** é um registrador de **64 bits**.



Acessando os dados - registradores

- **Registradores *AMD64***

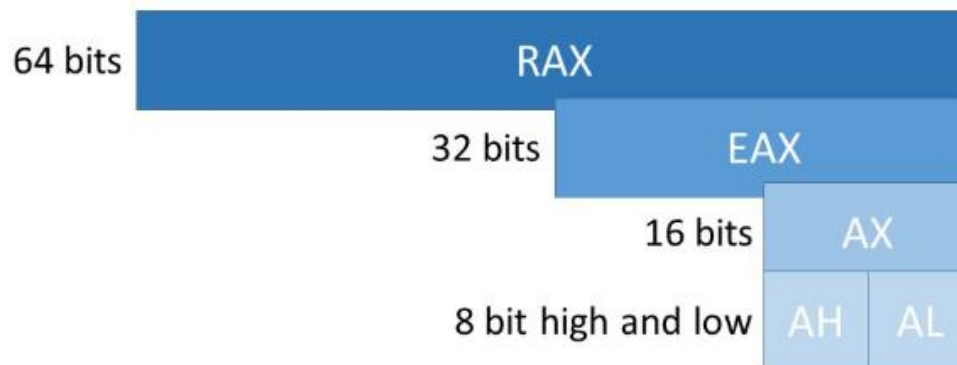
- Na arquitetura x64 registradores conhecidos **EAX, EBX, ECX, EDC, ESI e EDI**, foram alterados, a sintaxe substituindo o 'E' pelo 'R', formando na versão estendida os **RAX, RBX, RCX, RDX, RSI e RDI**, num total de 16 registradores de propósito geral, com a inclusão dos novos registradores nomeados **R8, R9, R10...R15**.



Acessando os dados - registradores

- Registradores *AMD64*

- De maneira similar, para RAX, RBX, RCX e RDX, o acesso do **lower 16 bits** é possível removendo o R no início (**AX para RAX**), e o **lower byte** desses mesmos trocando o X pelo L (**AL para AX**), e o **higher byte** do low 16 bits usando o H (**AH para AX**).



Registadores x86-64

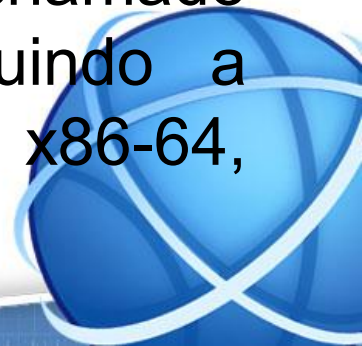
64bits	32bits	16bits	8bits
r0 (rax)	r0d (eax)	r0w (ax)	r0l (al)
r1 (rcx)	r1d (ecx)	r1w (cx)	r1l (cl)
r2 (rdx)	r2d (edx)	r2w (dx)	r2l (dl)
r3 (rbx)	r3d (ebx)	r3w (bx)	r3l (bl)
r4 (rsp)	r4d (esp)	r4w (sp)	
r5 (rbp)	r5d (ebp)	r5w (bp)	
r6 (rsi)	r6d (esi)	r6w (si)	
r7 (rdi)	r7d (edi)	r7w (di)	
r8 ~ r15			



Registradores x86-64

- Registradores

- Um registrador importante, mas que não está listado acima é o **%rip** (*instruction pointer*). Ele armazena sempre o endereço da próxima instrução a ser executada.
- Todas as arquiteturas baseadas na *arquitetura von Neumann* tem um registrador equivalente. Na maioria delas, este registrador é chamado “*program counter*”, ou **PC**, mas seguindo a nomenclatura dos processadores da linha x86-64, eles são chamados *instruction pointer*.



Acessando os dados - registradores

E_a = indica o registrador a ;

$R[E_a]$ = indica o valor contido no registrador a ;

Type	Form	Operand value	Name
Immediate	$\$Imm$	Imm	Immediate
Register	E_a	$R[E_a]$	Register
Memory	Imm	$M[Imm]$	Absolute
Memory	(E_a)	$M[R[E_a]]$	Indirect
Memory	$Imm(E_b)$	$M[Imm + R[E_b]]$	Base + displacement
Memory	(E_b, E_i)	$M[R[E_b] + R[E_i]]$	Indexed
Memory	$Imm(E_b, E_i)$	$M[Imm + R[E_b] + R[E_i]]$	Indexed
Memory	$(, E_i, s)$	$M[R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(, E_i, s)$	$M[Imm + R[E_i] \cdot s]$	Scaled indexed
Memory	(E_b, E_i, s)	$M[R[E_b] + R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(E_b, E_i, s)$	$M[Imm + R[E_b] + R[E_i] \cdot s]$	Scaled indexed



Acessando os dados – Memória

3. Endereçamento indireto:

- No modo de endereçamento indireto, um dos parâmetros indica uma referência indireta a memória (normalmente por um registrador entre parênteses).
- Por exemplo, a instrução **movq (%rbx), %rax** copia o conteúdo do endereço contido no registrador %rbx para o registrador %rax, algo como:
 - $\%rax = M[\%rbx]$
 - Onde $M[\%rbx]$ indica o conteúdo de memória indexado por %rbx.



Acessando os dados – Memória

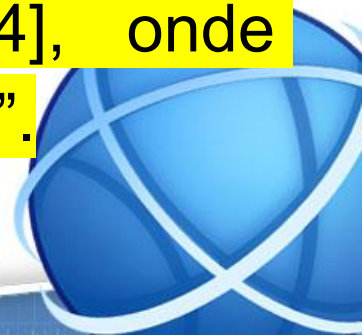
- **Endereçamento indireto:**
 - **Dupla indireção: ((%reg)) não é válido.**
 - **Somente um operando pode ser indireto.**
 - Por isso, `movq (%rbx), (%rax)` **não é válido.**
 - É possível usar endereçamento indireto com rótulos, ou seja, **são válidas as instruções do tipo `movq (A), %rax`.**



Acessando os dados – Memória

4. Endereçamento indexado:

- No endereçamento indexado, a instrução usa um “**endereço base**” e um deslocamento.
- Um exemplo é a instrução
 - **movq *vetorA*(,%rdi, 4), %rbx**,
que usa “***vetorA***” como base e %rdi x 4 como deslocamento.
- A instrução pode ser melhor entendida pela fórmula:
 - **%rbx = Memória[&***vetorA*** + %rdi*4]**, onde &***vetorA*** indica o endereço de “***vetorA***”.



Acessando os dados – Memória

$\$Imm$ = indica um valor imediato Imm ;

E_a = indica o registrador a ;

$R[E_a]$ = indica o valor contido no registrador a ;

Type	Form	Operand value	Name
Immediate	$\$Imm$	Imm	Immediate
Register	E_a	$R[E_a]$	Register
Memory	Imm	$M[Imm]$	Absolute
Memory	(E_a)	$M[R[E_a]]$	Indirect
Memory	$Imm(E_b)$	$M[Imm + R[E_b]]$	Base + displacement
Memory	(E_b, E_i)	$M[R[E_b] + R[E_i]]$	Indexed
Memory	$Imm(E_b, E_i)$	$M[Imm + R[E_b] + R[E_i]]$	Indexed
Memory	$(, E_i, s)$	$M[R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(, E_i, s)$	$M[Imm + R[E_i] \cdot s]$	Scaled indexed
Memory	(E_b, E_i, s)	$M[R[E_b] + R[E_i] \cdot s]$	Scaled indexed
Memory	$Imm(E_b, E_i, s)$	$M[Imm + R[E_b] + R[E_i] \cdot s]$	Scaled indexed



Registradores x86-64

- Registradores com funcionalidades específicas

Registrador	Uso
%rax	Temporário, 1 ^o retorno procedimento
%rbx	Temporário
%rcx	4 ^o parâmetro inteiro
%rdx	3 ^o parâmetro inteiro
%rsp	<i>stack-pointer</i>
%rbp	<i>base-pointer</i>
%rsi	2 ^o parâmetro inteiro
%rdi	1 ^o parâmetro inteiro
%r8	5 ^o parâmetro inteiro
%r9	6 ^o parâmetro inteiro
%r10	Temporário
%r11	Temporário
%r12-r15	Temporário



Linguagem Assembly

Todas as instruções contendo mais de um operando, guardam seu resultado em op2.

Para especificar o tamanho do(s) operando(s), acrescentar sufixo na instrução: B,W,L ou Q.

• Sufixo	Nome	Tamanho
– B	BYTE	1 byte (8 bits)
– W	WORD	2 bytes (16 bits)
– L	LONG (DOUBLE)	4 bytes (32 bits)
– Q	QUAD WORD	8 bytes (64 bits)
– S	SINGLE FLOAT	4 bytes (32 bits)
– D	DOUBLE FLOAT	8 bytes (64 bits)



Linguagem Assembly

- Instrução mov s (origem) para d (destino) de acordo com seu tamanho:
- mov[b | w | l | q] s , d move s to d
- Exemplos e combinações de parâmetros:

movq	reg_1, reg_2	$reg_2 \leftarrow reg_1$
movq	reg, mem	$M[mem] \leftarrow reg$
movq	mem, reg	$reg \leftarrow M[mem]$
movq	$imm64, reg$	$reg \leftarrow imm64$



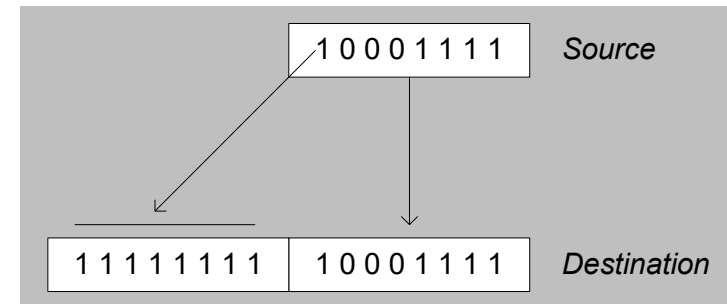
Linguagem Assembly

- Instrução mov s (origem) para d (destino) de acordo com seu tamanho:
- `mov[ b | w | l | q ] s , d` move s to d
 - `movb $10,%ah` ; para *byte 8bits*
 - `movw %bx,%ax` ; para *word 16bits*
 - `movl $ebx,%eax` ; para *long(double) 32bits*
 - `movq $10,%rax` ; para *qword 64bits*



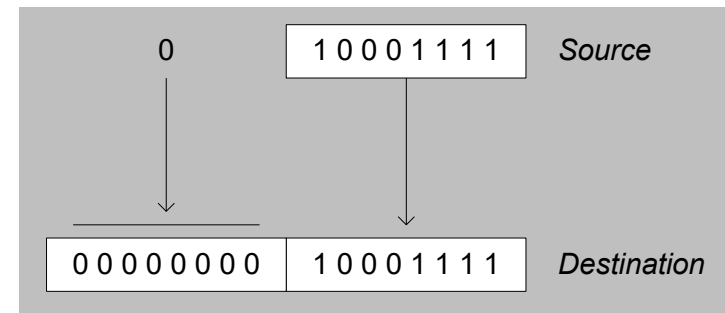
Linguagem Assembly

- Instrução mov s (origem) para d (destino) de acordo com seu tamanho:
- `movs[ b | w | l | q ] s , d` move s to d
- `movs com sinal`
 - `movs[bw|bl|bq|wl|wq|lq]` s, d
 - **bw** *byte* para *word*
 - **bl** *byte* para *long* (dword - double)
 - **bq** *byte* para *qword*
 - **wl** *word* para *long*
 - **wq** *word* para *qword*
 - **lq** *long* para *qword*



Linguagem Assembly

- Instrução mov s (origem) para d (destino) de acordo com seu tamanho:
- `movz[ b | w | l | q ] s , d` move s to d
- `movz` com extensão zero
 - `movz[bw|bl|bq|wl|wq|lq] s, d`
 - **bw byte** para **word**
 - **bl byte** para **long** (dword - double)
 - **bq byte** para **qword**
 - **wl word** para **long**
 - **wq word** para **qword**
 - **lq long** para **qword**



Linguagem Assembly

- Instruções aritméticas: **add [b | w | l | q] :**

- Combinações de parâmetros:

```
add <reg>, <reg>
add <mem>, <reg>
add <reg>, <mem>
add <con>, <reg>
add <con>, <mem>
```

- **Exemplos e combinações de parâmetros:**

addq	reg_1, reg_2	$reg_2 \leftarrow reg_2 + reg_1$
addq	reg, mem	$M[mem] \leftarrow M[mem] + reg$
addq	$imm64, reg$	$reg \leftarrow reg + imm64$
addq	$imm64, mem$	$M[mem] \leftarrow M[mem] + imm64$
addq	mem, reg	$reg \leftarrow reg + M[mem]$



Linguagem Assembly

- Instruções aritméticas: **sub [b | w | l | q] :**

- Combinações de parâmetros:

```
sub <reg>, <reg>
sub <mem>, <reg>
sub <reg>, <mem>
sub <con>, <reg>
sub <con>, <mem>
```

- **Exemplos e combinações de parâmetros:**

subq	<i>reg₁, reg₂</i>	$reg_2 \leftarrow reg_2 - reg_1$
subq	<i>reg, mem</i>	$M[mem] \leftarrow M[mem] - reg$
subq	<i>imm64, reg</i>	$reg \leftarrow reg - imm64$
subq	<i>imm64, mem</i>	$M[mem] \leftarrow M[mem] - imm64$
subq	<i>mem, reg</i>	$reg \leftarrow reg - M[mem]$



Linguagem Assembly

- Instruções aritméticas: `mul [ b | w | l | q ]`
 - Realiza operação de multiplicação sem sinal;
 - Não tem o mesmo formato que as operações aritméticas anteriores porque a multiplicação pode gerar um número que tem até o dobro de *bits* que os operandos.

`MUL reg/mem8`

`MUL reg/mem16`

`MUL reg/mem32`

`MUL reg/mem64`



Linguagem Assembly

- Instruções aritméticas: `mul [ b | w | l | q ]`
 - Realiza operação de multiplicação sem sinal;
 - MUL reg|mem
 - Se o operando tem 8 bits, por exemplo,
MUL %bh
então o comando equivale a
 $\%ax \leftarrow \%al \times \%bl$
Ou seja, o operando é sempre multiplicado pelo valor em %al
e o resultado é armazenado em %ax.



Linguagem Assembly

- Instruções aritméticas: `mul [ b | w | l | q ]`
 - Realiza operação de multiplicação sem sinal;
 - MUL reg|mem

Se o operando tem 16 bits, por exemplo,

MUL %bx

então o comando equivale a

`%dx:%xax ← %ax × %bx`

Ou seja, o operando é sempre multiplicado pelo valor em %ax e o resultado de 32 bits é armazenado em 2 registradores de 16 bits:

os 16 primeiros bits em %ax e os 16 últimos em %dx.



Linguagem Assembly

- Instruções aritméticas: `mul [ b | w | l | q ]`
 - Realiza operação de multiplicação sem sinal;
 - MUL reg|mem
 - Se o operando tem 32 bits, por exemplo,
MUL %ebx
então o comando equivale a
 $\%edx:\%eax \leftarrow \%eax \times \%ebx$
Ou seja, o operando é sempre multiplicado pelo valor em %eax e o resultado de 64 bits é armazenado em 2 registradores de 32 bits: os 32 primeiros bits em %eax e os 32 últimos em %edx



Linguagem Assembly

- Instruções aritméticas: **imul [b | w | l | q]**:
 - Realiza operação de multiplicação com sinal;
 - Combinações de parâmetros:

```
imul <reg32>, <reg32>
imul <mem>, <reg32>
imul <con>, <reg32>, <reg32>
imul <con>, <mem>, <reg32>
```

- **Exemplos e combinações de parâmetros:**

imulq	reg_1, reg_2	$reg_2 \leftarrow reg_2 \times reg_1$
imulq	mem, reg	$reg \leftarrow reg \times M[mem]$
imulq	$imm64, reg$	$reg \leftarrow reg \times imm64$



Linguagem Assembly

DIV reg/mem8

- Instruções aritméticas: **div [b | w | l | q]**

Division Size	Dividend	Divisor	Quotient	Remainder	Maximum Quotient
Word/byte	AX	reg/mem8	AL	AH	255
Doubleword/word	DX:AX	reg/mem16	AX	DX	65,535
Quadword/doubleword	EDX:EAX	reg/mem32	EAX	EDX	$2^{32} - 1$
Double quadword/quadword	RDX:RAX	reg/mem64	RAX	RDX	$2^{64} - 1$

DIV reg/mem16

DIV reg/mem32

- Funciona de forma inversa ao MUL.
- DIV é válida apenas para a divisão de números inteiros sem sinal.

DIV reg/mem64



Linguagem Assembly

- Instruções aritméticas: `idiv [ b | w | l | q ] s`

Division Size	Dividend	Divisor	Quotient	Remainder	Quotient Range
Word/byte	AX	reg/mem8	AL	AH	-128 to +127
Doubleword/word	DX:AX	reg/mem16	AX	DX	-32,768 to +32,767
Quadword/doubleword	EDX:EAX	reg/mem32	EAX	EDX	-2^{31} to $2^{31}-1$
Double quadword/quadword	RDX:RAX	reg/mem64	RAX	RDX	-2^{63} to $2^{63}-1$

- Exemplos e combinações de parâmetros:

<code>idivq</code>	<code>reg</code>	$\%rax \leftarrow \%rdx : \%rax \text{ DIV } reg$ $\%rdx \leftarrow \%rdx : \%rax \text{ MOD } reg$
<code>idivq</code>	<code>mem</code>	$\%rax \leftarrow \%rdx : \%rax \text{ DIV } M[mem]$ $\%rdx \leftarrow \%rdx : \%rax \text{ MOD } M[mem]$



Linguagem Assembly

- Instruções estende sinal: `c [ b | w | l | q ] [ w | l | q ]`
É `c [ b | w | d | q ] [ w | d | q | to ]`
 - `cbw` 8 → 16 bits
 - `cwd` 16 → 32 bits
 - `cdq` 32 → 64 bits
 - `cqto` 64 → 128 bits
 - `cqto` copia o sinal de `%rax` para `%rdx`
 - Forma o dividendo para `idiv`
 - Essencial para divisões com números negativos

```
movq $-10, %rax
cqto
movq $3, %rbx
idivq %rbx
```



Linguagem Assembly

- Instruções aritméticas: `inc [ b | w | l | q ] op`
 - Incremento
- Instruções aritméticas: `dec [ b | w | l | q ] op`
 - Decremento
 - Exemplos e combinações de parâmetros:

```
inc <reg>
```

```
inc <mem>
```

```
dec <reg>
```

```
dec <mem>
```



Linguagem Assembly

- Instruções lógicas: **and [b | w | l | q] s, d**
 - Realiza operação lógica AND;
 - **Exemplos e combinações de parâmetros:**

andq	<i>reg₁, reg₂</i>	<i>reg₂ ← reg₂ AND reg₁</i>
andq	<i>reg, mem</i>	<i>M[mem] ← M[mem] AND reg</i>
andq	<i>imm64, reg</i>	<i>reg ← reg AND imm64</i>
andq	<i>imm64, mem</i>	<i>M[mem] ← M[mem] AND imm64</i>
andq	<i>mem, reg</i>	<i>reg ← reg AND M[mem]</i>



Linguagem Assembly

- Instruções lógicas: **or [b | w | l | q] s, d**
 - Realiza operação lógica OR;
 - **Exemplos e combinações de parâmetros:**

<code>orq</code>	<code>reg₁, reg₂</code>	<code>reg₂ ← reg₂ OR reg₁</code>
<code>orq</code>	<code>reg, mem</code>	<code>M[mem] ← M[mem] OR reg</code>
<code>orq</code>	<code>imm64, reg</code>	<code>reg ← reg OR imm64</code>
<code>orq</code>	<code>imm64, mem</code>	<code>M[mem] ← M[mem] OR imm64</code>
<code>orq</code>	<code>mem, reg</code>	<code>reg ← reg OR M[mem]</code>



Linguagem Assembly

- Instruções lógicas: `xor [ b | w | l | q ] s, d`
 - Realiza operação lógica XOR;
 - Exemplos e combinações de parâmetros:

<code>xorq</code>	<code>reg₁, reg₂</code>	$reg_2 \leftarrow reg_2 \text{ XOR } reg_1$
<code>xorq</code>	<code>reg, mem</code>	$M[mem] \leftarrow M[mem] \text{ XOR } reg$
<code>xorq</code>	<code>imm64, reg</code>	$reg \leftarrow reg \text{ XOR } imm64$
<code>xorq</code>	<code>imm64, mem</code>	$M[mem] \leftarrow M[mem] \text{ XOR } imm64$
<code>xorq</code>	<code>mem, reg</code>	$reg \leftarrow reg \text{ XOR } M[mem]$



Linguagem Assembly

- Instruções lógicas: `not [ b | w | l | q ] op`
 - Complemento de 1
 - Exemplos e combinações de parâmetros:

<code>notq reg</code>		<code>reg ← ~ reg</code>
<code>notq mem</code>		<code>mem ← ~ mem</code>



Linguagem Assembly

- Instruções lógicas: `neg [ b | w | l | q ] op`
 - Realização a negação do valor especificado – inverte o sinal do valor;
 - Parâmetros:

`neg <reg>`

`neg <mem>`

- **Exemplo:**

```
negl %EBX
```

```
#   inverte o sinal de EBX para -EBX
```

```
#   inverte o sinal de -EBX para EBX
```



Linguagem Assembly

- Instruções lógicas: shl **[b | w | l | q] s, d**
 - Realiza operação de movimentação de bits a esquerda;
 - **Exemplos e combinações de parâmetros:**

shlq	<i>imm64, reg</i>		$reg \leftarrow reg \ll imm64$
shlq	<i>imm64, mem</i>		$mem \leftarrow mem \ll imm64$



Linguagem Assembly

- Instruções lógicas: shr **[b | w | l | q] s, d**
 - Realiza operação de movimentação de bits a direita;
 - **Exemplos e combinações de parâmetros:**

shrq	<i>imm64, reg</i>	$reg \leftarrow reg \gg imm64$ (logical shift)
shrq	<i>imm64, mem</i>	$mem \leftarrow mem \gg imm64$



Linguagem Assembly

- Instruções : `lea[b | w | l | q]s, d`
 - Retorna o endereço efetivo da rótulo especificado;
 - **Exemplo e combinação de parâmetros:**

`leaq mem, reg` | $reg \leftarrow mem$ (load effective address)



Referência

